



UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE

INGENIERÍA DE SOFTWARE

Análisis y Diseño de Software
30724

**Modelado de casos de uso de nivel 0 y apoyo al análisis
orientado a objetos con PlantUML**

CRUD de Estudiante

Presenta:

Cuzco Yamasca Alex Darío

Domínguez Aguilera Pablo Alejandro

Alvarado Palacios Dylan Alexander

Docente:

Jenny Alexandra Ruiz Robalino

Fecha:

05 de Mayo de 2026



Índice

1. Objetivos	3
1.1. Objetivo general	3
1.2. Objetivos específicos	3
2. Desarrollo	3
2.1. Requisitos funcionales Nivel 0	4
2.2. Diagrama de casos de uso	4
2.3. Especificación breve de casos de uso	8
2.4. Puente hacia clases de análisis	9
2.5. Diagrama de clases de análisis	12
2.6. Matriz de Trazabilidad	15
3. Conclusiones	16
4. Recomendaciones	16

1. Objetivos

1.1. Objetivo general

Modelar los requisitos funcionales de nivel 0 y el diagrama de casos de uso para el software EVENTUAL, asegurando que las funcionalidades de acceso al software, consulta del calendario de eventos y la propuesta de eventos trazables y verificables, con el fin de garantizar una implementación eficiente y coherente con los requerimientos del usuario.

1.2. Objetivos específicos

- Identificar los requisitos funcionales de nivel 0, como el acceso al software y la gestión de eventos, para asegurar la trazabilidad entre las necesidades de los socios y las funcionalidades del sistema.
- Modelar la interacción de los actores (Socio, Presidente, Tesorero y Secretario) con el sistema mediante diagramas de casos de uso en PlantUML, estableciendo los límites del software y las relaciones de dependencia necesarias.
- Establecer los criterios de aceptación y las restricciones técnicas del software, garantizando que la solución sea compatible con dispositivos móviles y cumpla con los estándares de seguridad y validación definidos en el SRS.

2. Desarrollo

Para el hacer el análisis de los 3 requisitos funcionales de nivel 0 para el sistema EVENTUAL se fundamenta en la transición de los requisitos abstractos hacia un modelo técnico funcional y verificable. Utilizando como base la Especificación de Requisitos de Software (SRS), se ha procedido a desglosar las necesidades del Club de Suboficiales retirados en unidades de trabajo denominadas Casos de Uso de nivel 0.

Este proceso sigue una metodología iterativa orientada a objetos, donde la trazabilidad es el eje principal. Para asegurar la calidad del diseño, se han seleccionado requisitos críticos que abarcan desde el control de seguridad (CU001: Acceso al software) hasta la interacción social y operativa del club (CU003 y CU004).

A continuación, se detalla el modelado realizado mediante la herramienta PlantUML, la cual permite representar gráficamente el límite del sistema, los roles de los actores (Socio, Presidente, Tesorero y Secretario) y su comportamiento esperado dentro de la solución móvil. Este análisis no sólo define qué hará el software, sino que establece los criterios de aceptación y las reglas de negocio, como los plazos de confirmación y las validaciones de identidad, que garantizarán el éxito de la implementación posterior en tecnologías como Flutter y Node.js.

2.1. Requisitos funcionales Nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El software debe permitir la autenticación de usuarios mediante cédula y contraseña.	Socio, Tesorero, Presidente, Secretario	Acceso al software
RF-03	El sistema debe mostrar una vista mensual de eventos programados con información detallada.	Socio, Tesorero, Presidente, Secretario	Consultar calendario de eventos
RF-04	Los socios deben poder proponer eventos (sociales o deportivos) mediante un formulario digital.	Socio	Proponer evento

2.2. Diagrama de casos de uso

El diagrama de casos de uso representa las interacciones entre los diferentes actores del sistema EVENTUAL (Socio, Presidente, Tesorero y Secretario) y las funcionalidades principales del software relacionadas con la gestión de eventos. Este modelo permite visualizar el alcance del sistema, identificando las responsabilidades de cada actor y describiendo cómo se llevarán a cabo procesos clave como el acceso al software, la consulta del calendario y la propuesta de eventos. Además, se incorporan relaciones como <<include>> y <<extend>> para reflejar comportamientos obligatorios y opcionales, garantizando una representación clara, estructurada y coherente con los requisitos definidos en el SRS.

```
@startuml
    Sistema de Planificación de Eventos

    left to right direction
    skinparam defaultFontName Arial
    skinparam defaultFontSize 11
    skinparam backgroundColor white
```

```
skinparam ActorBorderColor #333333
skinparam ActorBackgroundColor white
skinparam UseCaseBorderColor #555555
skinparam UseCaseBackgroundColor #EEEEEE
skinparam ArrowColor #333333
skinparam NoteBackgroundColor #FFFFC0
skinparam NoteBorderColor #999999
skinparam NoteFontSize 10
skinparam PackageBorderColor #333333
skinparam PackageBackgroundColor white
skinparam PackageFontStyle bold

title Sistema de Planificación de Eventos

actor "Usuario" as usuario
actor "Socio" as socio
actor "Tesorero" as tesorero
actor "Presidente" as presidente

rectangle "Casos de Uso Principales" as CUP {
    usecase "CU-001\nAcceso al Software" as CU001
    usecase "CU-003\nConsultar Calendario\nde Eventos" as CU003
    usecase "CU-004\nProponer Evento" as CU004
}

rectangle "Procesos Internos" as PI {
    usecase "Validar credenciales" as VC
    usecase "Generar sesión" as GS
    usecase "Filtrar eventos" as FE
    usecase "Validar propuesta" as VP
    usecase "Notificar directiva" as ND
}

note right of VC
    **Ingreso con:**
    * Cédula
    * Contraseña
end note

note right of GS
    **Permite visualizar:**
    * Eventos mensuales
```

```
* Tipo de evento
* Detalle del evento
end note

note right of VP
  **Datos de propuesta:**
  * Tipo
  * Descripción
  * Fecha sugerida
  * Justificación
end note

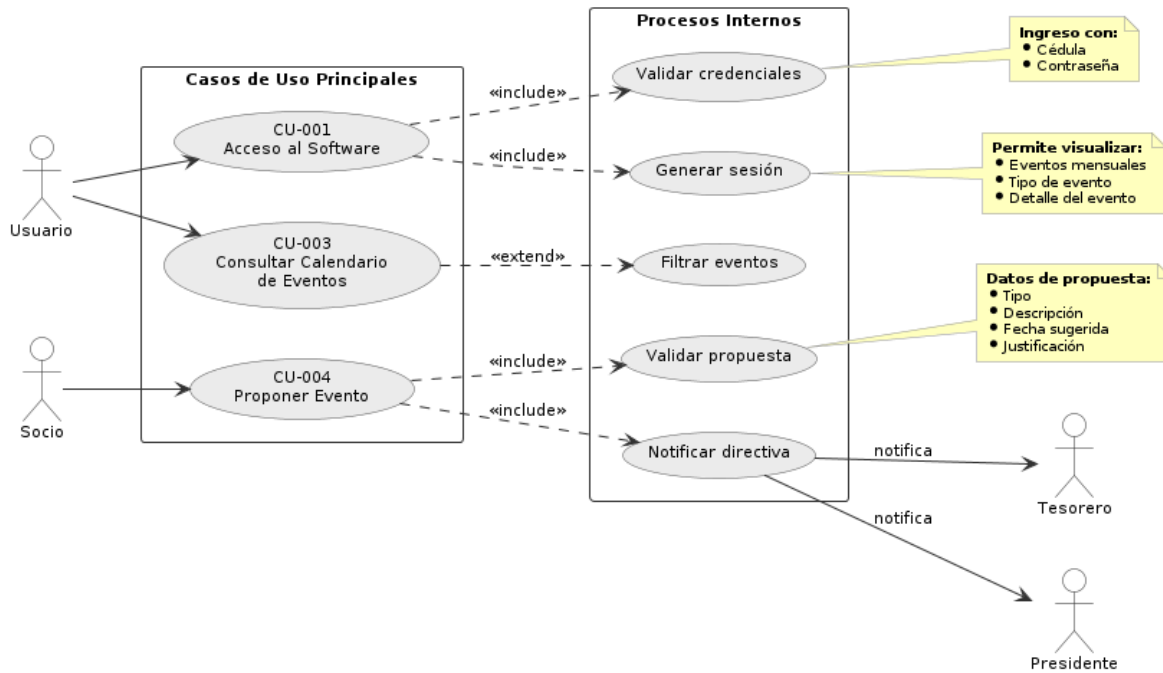
' Relaciones actores izquierda
usuario --> CU001
usuario --> CU003
socio --> CU004

' Include / Extend
CU001 ..> VC : <<include>>
CU001 ..> GS : <<include>>
CU003 ..> FE : <<extend>>
CU004 ..> VP : <<include>>
CU004 ..> ND : <<include>>

' Notificaciones actores derecha
ND --> tesorero : notifica
ND --> presidente : notifica

@enduml
```

Sistema de Planificación de Eventos



2.3. Especificación breve de casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
CU-001: Acceso al software	Socio, Presidente, Tesorero, Secretario	El usuario debe estar registrado en el sistema	El usuario ingresa su cédula y contraseña → el sistema valida credenciales → si son correctas, genera sesión y permite el acceso → si son incorrectas, muestra error y controla intentos	Usuario autenticado accede al sistema según su rol

CU-003: Consultar calendario de eventos	Socio, Presidente, Tesorero, Secretario	El usuario debe estar autenticado	El usuario accede al módulo de calendario → el sistema muestra eventos programados → el usuario puede seleccionar un evento para ver detalles, filtrar por tipo o navegar entre fechas	Visualización correcta de eventos con información detallada
CU-004: Proponer evento	Socio	El usuario debe estar autenticado	El socio accede a la opción “Proponer Evento” → completa formulario (tipo y descripción) → envía la propuesta → el sistema valida datos y registra la propuesta como pendiente → notifica a la directiva	Evento registrado en estado “Pendiente” y notificación enviada

2.4. Puente hacia clases de análisis

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante	Sintaxis PlantUML sugerida
----------------------	----------------------------------	---	---

Boundary (Interfaz)	Representa la interacción entre el usuario y el sistema. Captura datos de entrada y muestra resultados.	Pantalla de inicio de sesión, formulario de propuesta de evento, vista de calendario	class "PantallaLogin" <<boundary>>
Control (Controlador)	Gestiona la lógica del sistema, coordina el flujo de los casos de uso y valida reglas de negocio.	ControlAcceso, ControlEvento, ControlAsistencia	class "ControlAcceso" <<control>>

Código para visualizar el puente entre casos de uso y clases de análisis:

```
@startuml
```

```
left to right direction
```

```
' ——— Actores —————
```

```
actor "Socio" as Socio
```

```
actor "Presidente" as Presidente
```

```
actor "Tesorero" as Tesorero
```

```
actor "Secretario" as Secretario
```

```
' ——— Boundary (Interfaz) —————
```

```
boundary "PantallaLogin" as LoginUI
```

```
boundary "PantallaCalendario" as CalendarUI
```

```
boundary "FormularioPropuestaEvento" as PropuestaUI
```

```
' ——— Control (Lógica) —————
```

```
control "ControlAcceso" as ControlAcceso
```

```
control "ControlEvento" as ControlEvento
```

```
' ——— Entity (Datos) —————
```

```
entity "Usuario" as Usuario
```

```
entity "Evento" as Evento
```

```
entity "PropuestaEvento" as Propuesta
```

```
' ——— Repository (Persistencia) —————
```

database "RepositorioUsuario" as RepoUsuario

database "RepositorioEvento" as RepoEvento

' — Interacciones Actor → Boundary —

Socio --> LoginUI

Presidente --> LoginUI

Tesorero --> LoginUI

Secretario --> LoginUI

Socio --> CalendarUI

Socio --> PropuestaUI

' — Flujo Boundary → Control —

LoginUI --> ControlAcceso : enviar credenciales

CalendarUI --> ControlEvento : solicitar eventos

PropuestaUI --> ControlEvento : enviar propuesta

' — Flujo Control → Entity —

ControlAcceso --> Usuario : validar datos

ControlEvento --> Evento : gestionar eventos

ControlEvento --> Propuesta : registrar propuesta

' — Flujo Control → Repository —

ControlAcceso --> RepoUsuario : consultar usuario

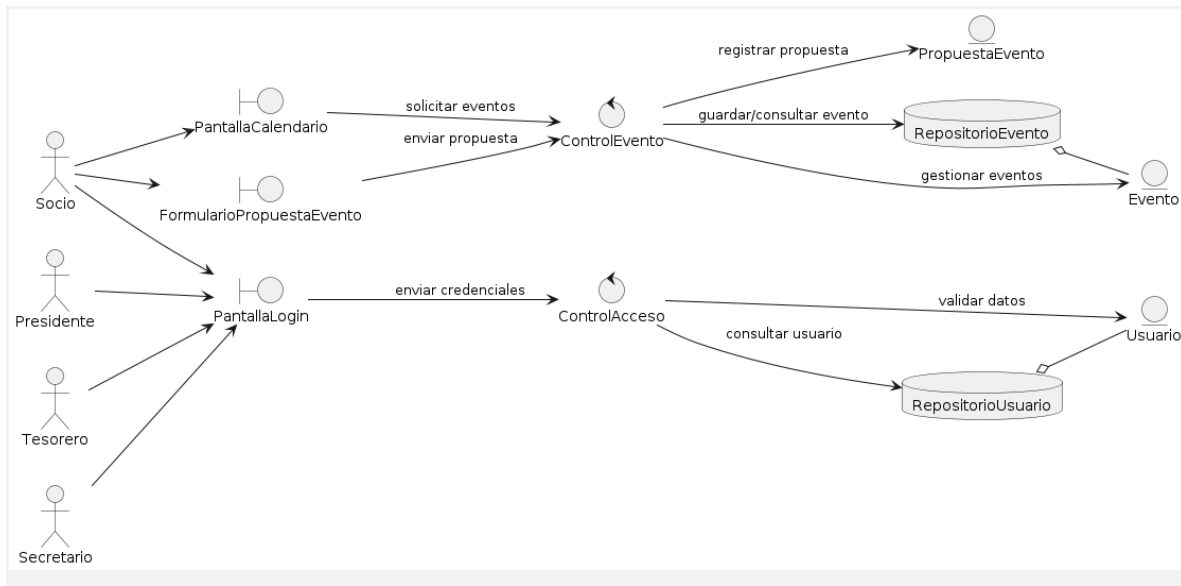
ControlEvento --> RepoEvento : guardar/consultar evento

' — Relaciones Repository → Entity —

RepoUsuario o-- Usuario

RepoEvento o-- Evento

@enduml



2.5. Diagrama de clases de análisis

El diagrama de clases de análisis del sistema EVENTUAL representa la estructura conceptual del software a partir de los elementos identificados en los casos de uso, organizados bajo el enfoque Boundary–Control–Entity–Repository. Este modelo permite definir claramente las responsabilidades de cada componente, donde las clases de tipo *boundary* gestionan la interacción con el usuario, las clases *control* coordinan la lógica del negocio, las clases *entity* representan la información persistente del sistema (como usuarios, eventos, propuestas y asistencias), y las clases *repository* se encargan de la gestión de datos. Esta estructura facilita la transición desde el análisis hacia el diseño orientado a objetos, asegurando coherencia, modularidad y trazabilidad con los requisitos definidos en la SRS.

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

' --- Boundary ---
class "PantallaLogin" <<boundary>> {
+ ingresarCredenciales()
+ mostrarMensaje(mensaje: String)
}

```

```

class "PantallaCalendario" <<boundary>> {
    + mostrarEventos()
    + filtrarEventos(tipo: String)
    + verDetalleEvento(id: String)
}

class "FormularioPropuestaEvento" <<boundary>> {
    + ingresarDatosEvento()
    + enviarPropuesta()
}

class "PantallaConfirmacionAsistencia" <<boundary>> {
    + confirmarAsistencia()
    + ingresarAcompañantes()
}

' — Control —————
class "ControlAcceso" <<control>> {
    + validarCredenciales(cedula: String, password: String): boolean
    + generarSesion(usuario: Usuario)
}

class "ControlEvento" <<control>> {
    + obtenerEventos(): List<Evento>
    + registrarPropuesta(propuesta: PropuestaEvento)
}

class "ControlAsistencia" <<control>> {
    + registrarAsistencia(asistencia: Asistencia)
    + actualizarParticipantes(evento: Evento)
}

' — Entity —————
class "Usuario" <<entity>> {
    - cedula: String
    - nombre: String
    - rol: String
}

class "Evento" <<entity>> {

```

```

- id: String
- nombre: String
- fecha: Date
- tipo: String
}

class "PropuestaEvento" <<entity>> {
- id: String
- tipo: String
- descripcion: String
- estado: String
}

class "Asistencia" <<entity>> {
- id: String
- confirmacion: boolean
- numAcompañantes: int
}

' — Repository —————
class "RepositorioUsuario" <<repository>> {
+ buscarUsuario(cedula: String): Usuario
}

class "RepositorioEvento" <<repository>> {
+ guardarEvento(evento: Evento)
+ listarEventos(): List<Evento>
}

class "RepositorioAsistencia" <<repository>> {
+ guardarAsistencia(asistencia: Asistencia)
}

' — Relaciones Boundary → Control —————
PantallaLogin --> ControlAcceso
PantallaCalendario --> ControlEvento
FormularioPropuestaEvento --> ControlEvento
PantallaConfirmacionAsistencia --> ControlAsistencia

' — Relaciones Control → Entity —————

```

```

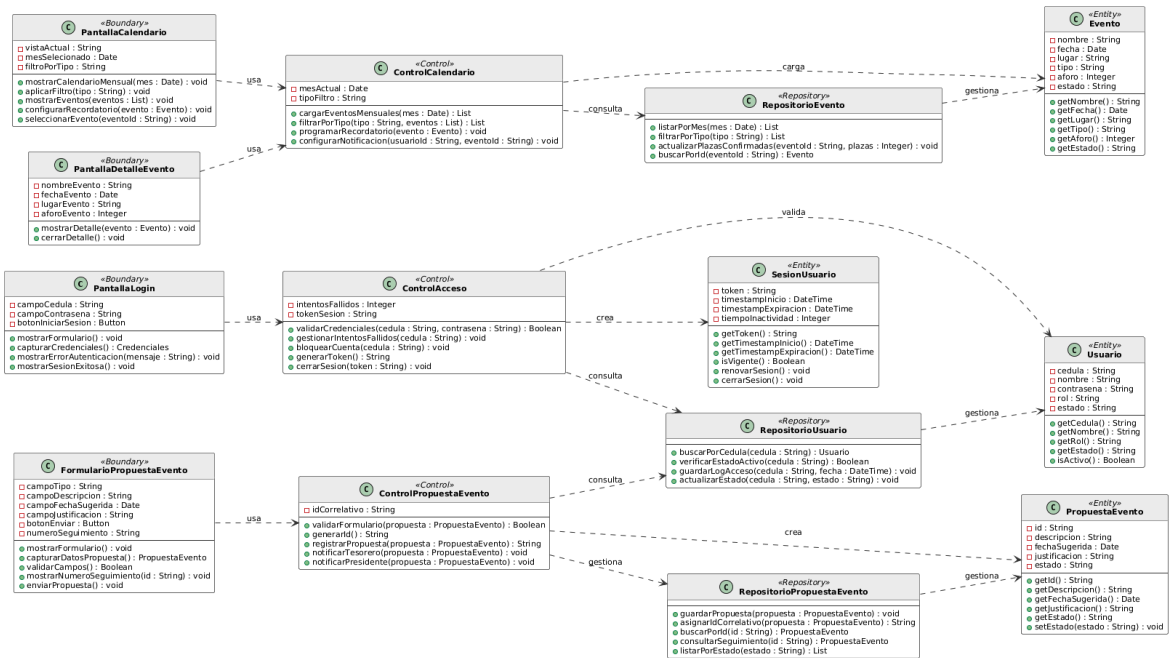
ControlAcceso --> Usuario
ControlEvento --> Evento
ControlEvento --> PropuestaEvento
ControlAsistencia --> Asistencia
ControlAsistencia --> Evento

' — Relaciones Control → Repository —
ControlAcceso --> RepositorioUsuario
ControlEvento --> RepositorioEvento
ControlAsistencia --> RepositorioAsistencia

' — Relaciones Repository → Entity —
RepositorioUsuario o-- Usuario
RepositorioEvento o-- Evento
RepositorioAsistencia o-- Asistencia

@enduml

```



2.6. Matriz de Trazabilidad

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
-------------	--------------	--------------------------	--------------------

Acceso al software (CU-001)	El usuario debe estar previamente registrado en el sistema	Ingresar cédula y contraseña → validar credenciales → si son correctas, generar sesión → si son incorrectas, mostrar error y controlar intentos	Usuario autenticado accede al sistema según su rol
Consultar calendario de eventos (CU-003)	El usuario debe estar autenticado	Acceder al módulo de calendario → visualizar eventos programados → seleccionar evento para ver detalles → opcionalmente filtrar o navegar entre fechas	Eventos mostrados correctamente con información detallada
Proponer evento (CU-004)	El usuario (Socio) debe estar autenticado	Acceder a “Proponer Evento” → ingresar datos (tipo y descripción) → validar información → registrar propuesta → notificar a la directiva	Propuesta registrada en estado “Pendiente” y notificación enviada

3. Conclusiones

- El modelado de los requisitos funcionales de nivel 0 permitió definir de manera clara los casos de uso principales del sistema EVENTUAL, asegurando que funcionalidades como el acceso al software, la consulta de eventos, la propuesta de eventos y la confirmación de asistencia respondan directamente a las necesidades de los actores involucrados.
- El uso de diagramas de casos de uso y su representación en PlantUML permitió estructurar adecuadamente las interacciones entre los actores (Socio, Presidente, Tesorero y Secretario) y el sistema, incorporando relaciones como <<include>> y <<extend>> para reflejar dependencias y comportamientos específicos de manera coherente.
- La trazabilidad establecida entre requisitos funcionales, casos de uso y clases de análisis (boundary, control, entity y repository) garantiza una transición ordenada hacia el diseño orientado a objetos, reduciendo inconsistencias y facilitando el desarrollo del software conforme a lo definido en la ERS.

4. Recomendaciones

- Refinar y validar continuamente los requisitos funcionales con los stakeholders, para asegurar que los casos de uso representen correctamente las necesidades reales del sistema EVENTUAL y evitar ambigüedades en etapas posteriores.
- Mantener un uso adecuado de las relaciones <<include>> y <<extend>> en los diagramas de casos de uso, evitando sobrecargar el modelo y asegurando claridad en la representación de la lógica del sistema.
- Actualizar de forma constante la matriz de trazabilidad conforme se realicen cambios en los requisitos o en el diseño, garantizando la coherencia entre los distintos artefactos del proyecto.
- Verificar la consistencia de las clases de análisis, especialmente en atributos, tipos de datos y responsabilidades, para asegurar una base sólida que facilite la implementación en tecnologías como Flutter, Node.js y bases de datos relacionales.